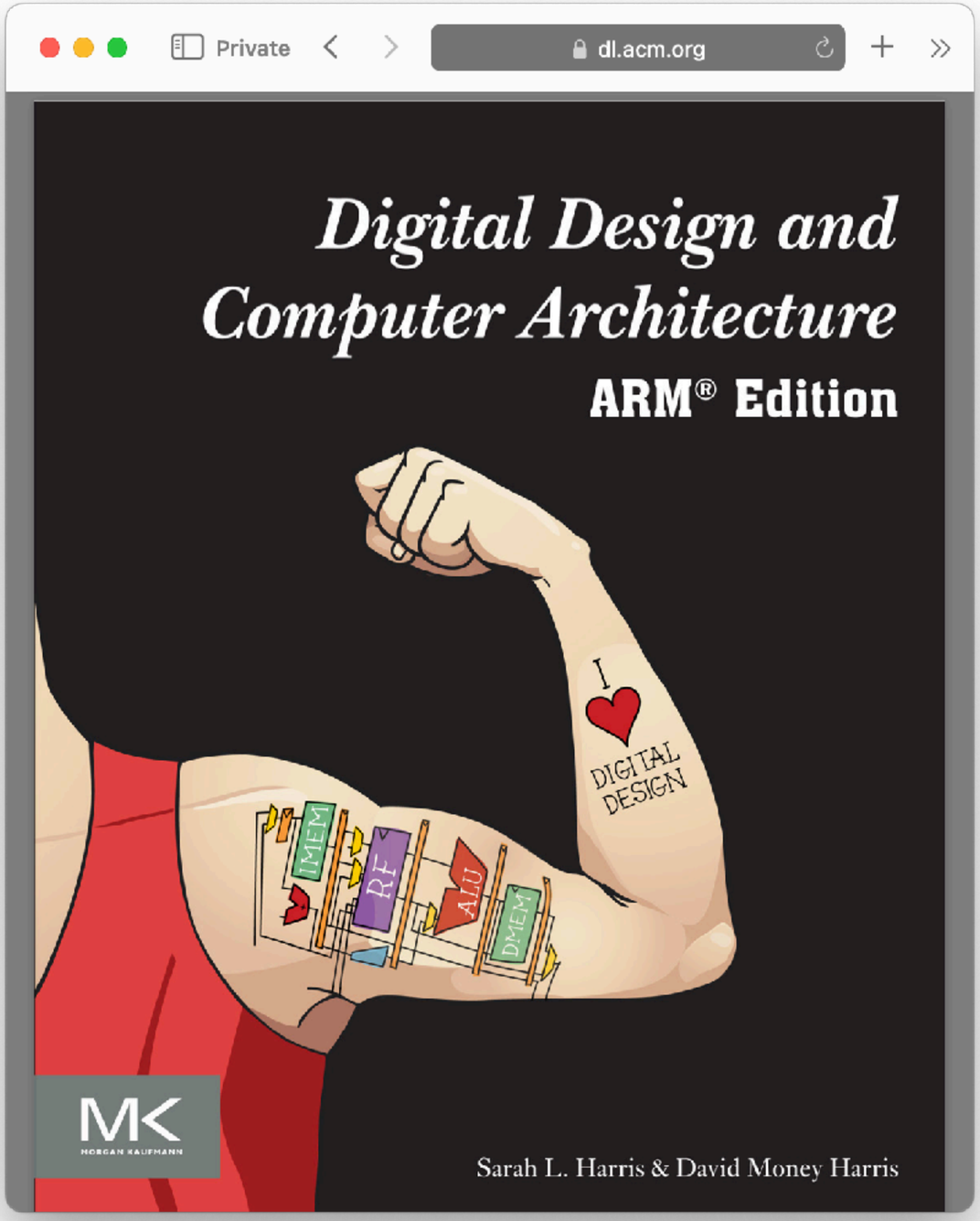


SystemVerilog

Alex Breuer

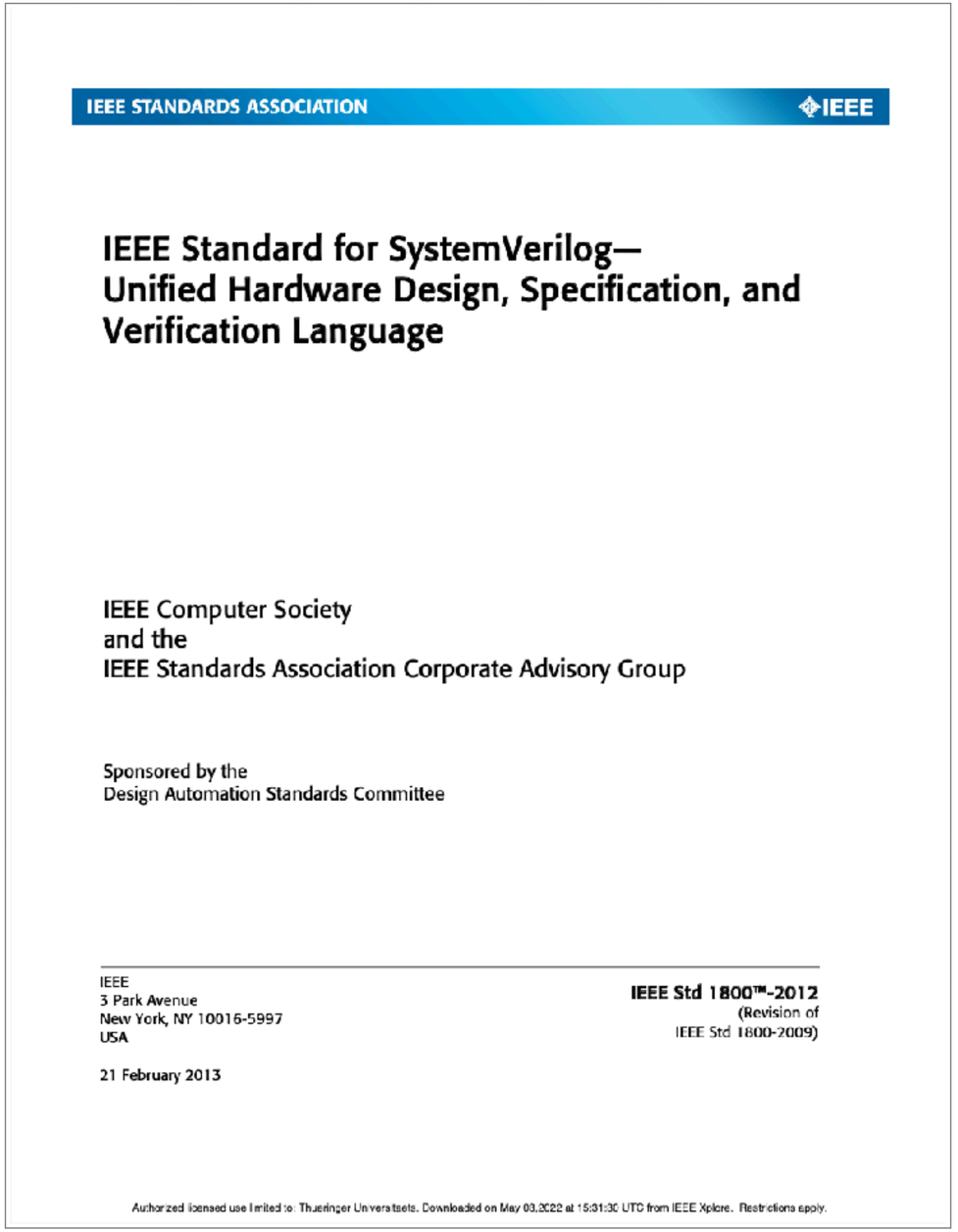
Introduction

- Until this point: Used CircuitVerse to study combinational logic
- Now: Hardware Description Language (HDL)
- HDLs are used in professional hardware development!
- Two big players: SystemVerilog and VHDL; we'll learn (some) SystemVerilog
- SystemVerilog: Digital modeling language and digital verification language
- This part follows up to some extend the textbook "Digital Design and Computer Architecture" by S. L. Harris and D. L. Harris; the book is available to all of you though ThULB: https://dl.acm.org/doi/pdf/10.5555/2815529



Hardware Description Language (HDL)

- Block of hardware is called a module
- Module in SystemVerilog might be described using different levels of abstractions
- We'll use SystemVerilog at the structural level first and later at the behavioral level:
  - Structural description: How is the module built from smaller pieces?
  - Behavioral description: What does the module do?
- HDLs are typically used for behavioral description. But: one still has to keep in mind the mapping to hardware!



IEEE STANDARDS ASSOCIATION IEEE

IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language

IEEE Computer Society
and the
IEEE Standards Association Corporate Advisory Group

Sponsored by the
Design Automation Standards Committee

IEEE
3 Park Avenue
New York, NY 10016-5997
USA

IEEE Std 1800™-2012
(Revision of
IEEE Std 1800-2009)

21 February 2013

Authorized licensed use limited to: Thueringer Universitaets. Downloaded on May 03,2022 at 15:31:30 UTC from IEEE Xplore. Restrictions apply.

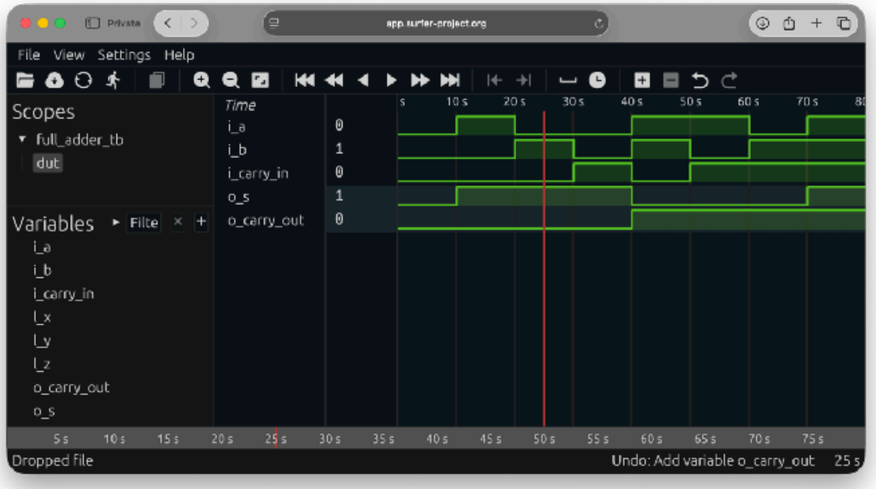
Source: https://ieeexplore.ieee.org/document/6469140

HDLs in Praxis

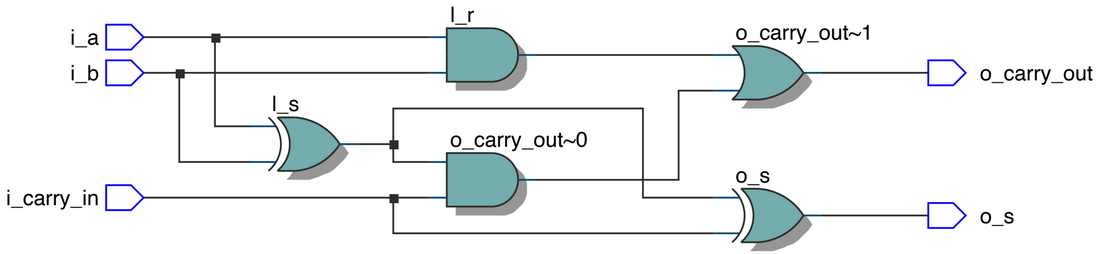
- Simulation:
  - Apply a selection of inputs to the circuit
  - Check the outputs
  - Allows us to look into our modules!
- Synthesis:
  - Use Computer-Aided Design (CAD) tool to lower the HDL description to optimized gates (not a unique mapping)
  - Result: netlist describing the hardware, i.e. a list of the logic gates and connecting wires

SystemVerilog assignment

FGPA-based assignments



Waveforms of a simulation of a full-adder module.



netlist visualization of a synthesized full-adder module.

Structural / Gate-level Modeling

SystemVerilog Modules

- Modules are implemented between `module` and `endmodule`
- Module's inputs / outputs use keywords `input logic` and `output logic`
- `Be aware of inherent parallelism!`

Primitive	#Inputs	#Outputs	Description
`and`	2 or more	1	AND gate
`nand`	2 or more	1	NAND gate
`or`	2 or more	1	OR gate
`nor`	2 or more	1	NOR gate
`xor`	2 or more	1	XOR gate
`xnor`	2 or more	1	XNOR gate
`buf`	1	1 or more	Buffer gate
`not`	1	1 or more	Inverter gate

High-Level Formula

$$C = F(A, B) = \bar{A} + B$$

SystemVerilog Module

```
/**
 * "My Module" which implements C = not(A) + B
 * using gate-level modeling.
 *
 * @param i_a input A.
 * @param i_b input B.
 * @param o_c output C.
 **/
module my_module( input  wire i_a,
                  input  wire i_b,
                  output wire o_c );
  wire l_x;

  not gate_0( l_x, i_a );
  or  gate_1( o_c, l_x, i_b );
endmodule
```

multi-line comment

input signals

name of the module

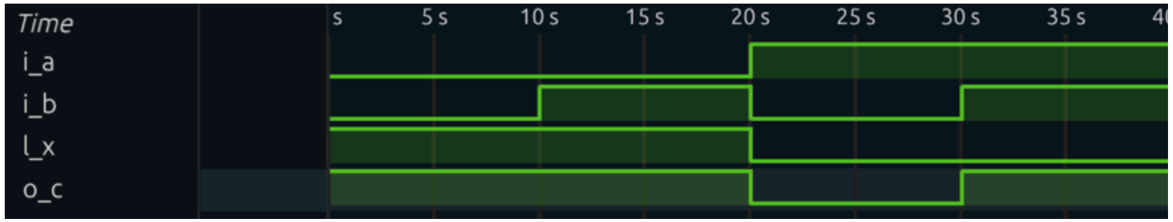
output signal

internal signal

inverter gate

OR gate

Simulation



SystemVerilog Modules

- Module might be composed of other modules
- General approach: Design complex systems hierarchically, i.e. start with very simple modules and use them to build more complex ones; then continue at the next level and so on..

Primitive	#Inputs	#Outputs	Description
`and`	2 or more	1	AND gate
`nand`	2 or more	1	NAND gate
`or`	2 or more	1	OR gate
`nor`	2 or more	1	NOR gate
`xor`	2 or more	1	XOR gate
`xnor`	2 or more	1	XNOR gate
`buf`	1	1 or more	Buffer gate
`not`	1	1 or more	Inverter gate

High-Level Formula

$$C = F(A, B) = \overline{\bar{A} + B}$$

SystemVerilog Module

```
/**
 * Another example module which implements C = not(not(A) + B)
 * using gate-level modeling.
 *
 * @param i_a input A.
 * @param i_b input B.
 * @param o_c output C.
 **/
module another_example( input  wire i_a,
                                    i_b,
                        output wire o_c );

  wire l_x;

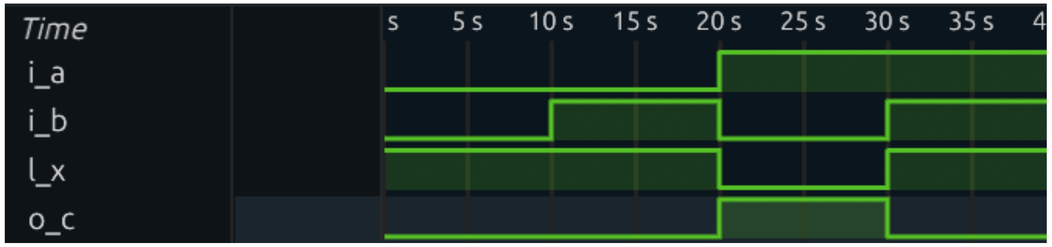
  my_module my_module( .i_a(i_a),
                       .i_b(i_b),
                       .o_c(l_x) );

  not m_gate_not( o_c, l_x );
endmodule
```

we may write "input wire" only once

instance of my_module

Simulation



SystemVerilog Modules

- Modules might use buses as inputs, as outputs or locally
- Be aware that the syntax is different from your standard arrays in C code!

High-Level Formula

$$C_{[1:0]} = A_{[1:0]}B_{[1:0]}, \quad C_{[3:2]} = A_{[1:0]} + B_{[1:0]}$$

SystemVerilog Module

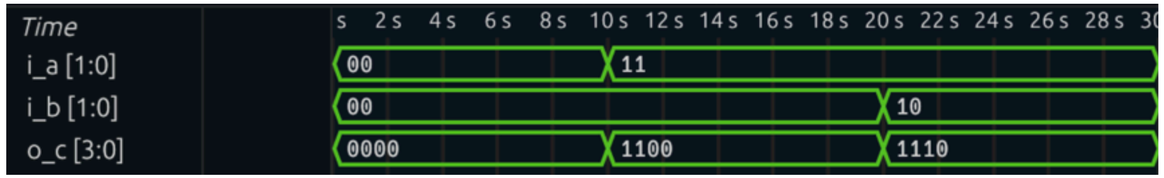
```
module example_with_buses( input  wire [1:0] i_a,
                           input  wire [1:0] i_b,
                           output wire [3:0] o_c );
  and gate_0( o_c[0:0],
              i_a[0:0],
              i_b[0:0] );

  and gate_1( o_c[1:1],
              i_a[1:1],
              i_b[1:1] );

  or  gate_2( o_c[2:2],
              i_a[0:0],
              i_b[0:0] );

  or  gate_3( o_c[3:3],
              i_a[1:1],
              i_b[1:1] );
endmodule
```

Simulation



Testbenches and Simulation

- We use testbenches to test our modules
- For few possible inputs we typically check every single one; for many options only a few selected ones
- Special commands allow us to:
  - Advance time
  - Check signals
  - Dump the inputs, outputs and the internal signals of our modules to disk

my_mdoule.sv

```
module my_module( input  wire i_a,
                  input  wire i_b,
                  output wire o_c );
  wire l_x;

  not gate_0( l_x, i_a );
  or  gate_1( o_c, l_x, i_b );
endmodule
```

my_module_tb.sv

```
module my_module_tb;
  logic l_a, l_b, l_c;

  my_module dut( .i_a( l_a ),
                 .i_b( l_b ),
                 .o_c( l_c ) );

  initial begin
    $dumpfile("dump_my_module.vcd");
    $dumpvars;

    l_a = 1'b0;
    l_b = 1'b0;
    #10;
    assert( l_c === 1'b1 );

    l_a = 1'b0;
    l_b = 1'b1;
    #10;
    assert( l_c === 1'b1 );

    l_a = 1'b1;
    l_b = 1'b0;
    #10;
    assert( l_c === 1'b0 );

    l_a = 1'b1;
    l_b = 1'b1;
    #10;
    assert( l_c === 1'b1 );

    $finish;
  end
endmodule
```

dumps the design state during simulation; output may be visualized through surfer

scope of the dump; here: dump everything

numeral base of the literal, here **b**inary

first digit of the number, here also the only one

check that the internal signal `l_c` is 1_2

bit-width of the literal here: 1

advance time by 10 units

finish simulation

Testbenches and Simulation

Compile

```
iverilog -Wall -Winfloop -g2012 my_module_tb.sv \
          my_module.sv -o my_module_tb
```

Simulate

```
./my_module_tb
```

View waveforms

```
https://app.surfer-project.org/
```

my_mdoule.sv

```
module my_module( input  wire i_a,
                  input  wire i_b,
                  output wire o_c );
  wire l_x;

  not gate_0( l_x, i_a );
  or  gate_1( o_c, l_x, i_b );
endmodule
```

my_module_tb.sv

```
module my_module_tb;
  logic l_a, l_b, l_c;

  my_module dut( .i_a( l_a ),
                 .i_b( l_b ),
                 .o_c( l_c ) );

  initial begin
    $dumpfile("dump_my_module.vcd");
    $dumpvars;

    l_a = 1'b0;
    l_b = 1'b0;
    #10;
    assert( l_c === 1'b1 );

    l_a = 1'b0;
    l_b = 1'b1;
    #10;
    assert( l_c === 1'b1 );

    l_a = 1'b1;
    l_b = 1'b0;
    #10;
    assert( l_c === 1'b0 );

    l_a = 1'b1;
    l_b = 1'b1;
    #10;
    assert( l_c === 1'b1 );

    $finish;
  end
endmodule
```

dumps the design state during simulation; output may be visualized through surfer

scope of the dump; here: dump everything

numeral base of the literal, here **b**inary

first digit of the number, here also the only one

check that the internal signal `l_c` is 1_2

bit-width of the literal here: 1

advance time by 10 units

finish simulation

Numbers

- Sized unsigned integer literals: `<size>'<base>value`
- `size` gives the size in bits
- `base` is the base, supported are:
  - b for binary
  - o for octal
  - d for decimal
  - h for hexadecimal
- `value` gives the value
- Underscores "_" are ignored but used for better readability

```
3'b101          = ?
8'b1101_0010    = ?
4'b101          = ?
5'd6            = ?
10'hf0          = ?
```

Numbers

- Sized unsigned integer literals:
  `<size>'<base>value`
- `size` gives the size in bits
- `base` is the base, supported are:
  - b for binary
  - o for octal
  - d for decimal
  - h for hexadecimal
- `value` gives the value
- Underscores "_" are ignored but used for better readability

```
3'b101          = 101
8'b1101_0010    = 11010010
4'b101          = 0101
5'd6            = 00110
10'hf0          = 0011110000
```

Delays

- We may specify the delays of our gates which are assumed during simulation:

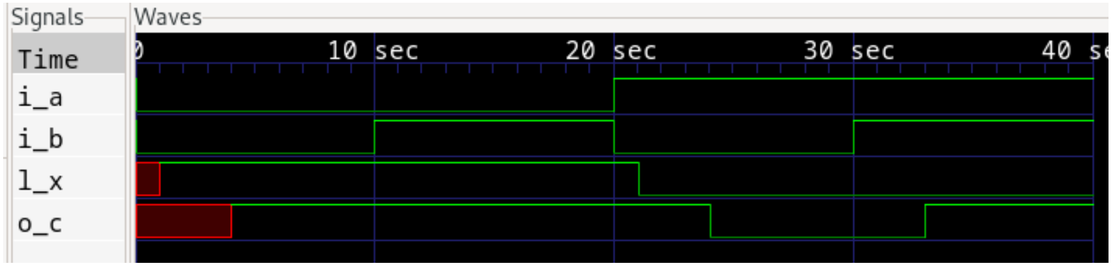
```
gate_type delay instance_name( outputs, inputs );
```

- Example:

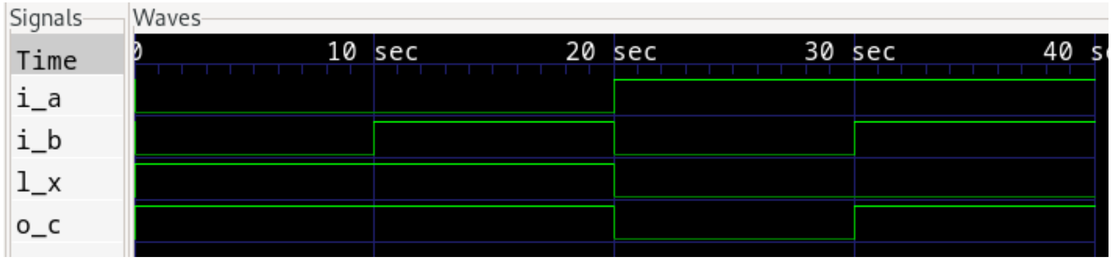
```
/**
 * "My Module" which implements C = not(A) + B
 * using gate-level modeling.
 *
 * @param i_a input A.
 * @param i_b input B.
 * @param o_c output C.
 **/
module my_module_with_delays( input  wire i_a,
                              input  wire i_b,
                              output wire o_c );

  wire l_x;

  not #1 m_gate_0( l_x, i_a );
  or  #3 m_gate_1( o_c, l_x, i_b );
endmodule
```



Simulation with delays



Simulation w/o delays

RTL-level Modeling

Continuous Assignments

- Keyword `assign` is used for continuous assignments which recompute the output whenever any of the inputs changes
- Bitwise operators may be used on one-bit inputs or multi-bit busses

Operator	Description
~	Bitwise NOT
&	Bitwise AND
~(a & b)	Bitwise NAND
^	Bitwise XOR
~^	Bitwise XNOR
|	Bitwise OR
~(a | b)	Bitwise NOR

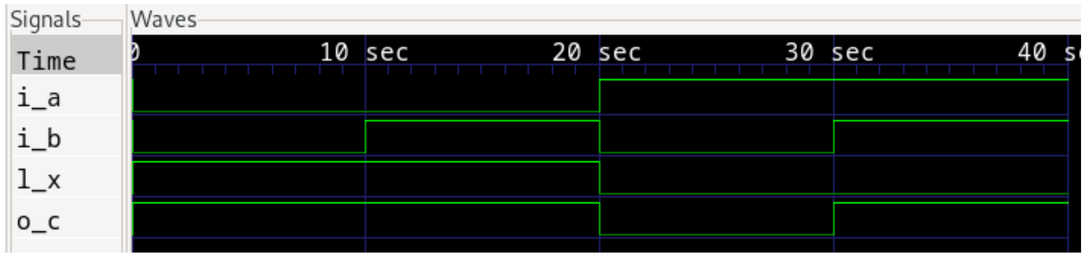
High-Level Formula

$$C = F(A, B) = \bar{A} + B$$

SystemVerilog Module

```
/**
 * “Example Module" which implements C = not(A) + B
 * using bitwise operators.
 *
 * @param i_a input A.
 * @param i_b input B.
 * @param o_c output C.
 **/
module simple_example_module( input  logic i_a,
                              input  logic i_b,
                              output logic o_c );
  assign o_c = ~i_a | i_b;
endmodule
```

Simulation



Shift and Arithmetic Operators

- Task 4. uses your 4-bit ripple carry adder (developed in 3.3)
- If you are concerned about the correctness of your implementation, you might use the module on the right
- Synthesizer will take care about lowering the module to gates and (most likely) not use a ripple-carry adder 😎

Operator	Description
<<	Left Shift
>>	Right Shift
+	Add
-	Subtract
*	Multiply
/	Divide
%	Modulus

```
/**
 * 4-bit adder.
 *
 * @param i_a input A.
 * @param i_b input B.
 * @param i_carry_in carry in.
 * @param o_s sum.
 * @param o_carry_out carry out.
 **/
module adder_4( input  logic [3:0] i_a,
                input  logic [3:0] i_b,
                input  logic       i_carry_in,
                output logic [3:0] o_s,
                output logic       o_carry_out );
  assign {o_carry_out, o_s} = i_a + i_b + i_carry_in;
endmodule
```

Conditional Operator

- Conditional operator is a ternary operator and allows to choose between two expressions
- Used extensively in RTL modeling (and in “regular” programming languages as well)

Operator	Description
`s ? x : y`	If s is true: select x If s is false: select y If s is unknown: special behavior (see literature)

```
/**
 * 8-bit multiplexer with two inputs.
 *
 * @param i_in0 first input.
 * @param i_in1 second input.
 * @param i_s control signal,
 *              if 0: in0 is output.
 *              if 1: in1 is output.
 * @param o_out output.
 **/
module mux_2( input  logic [7:0] i_in0,
                                 i_in1,
              input  logic       i_s,
              output logic [7:0] o_out );
  assign o_out = i_s ? i_in1 : i_in0;
endmodule
```

Parameterized Modules

- Until now: all of our modules had fixed lengths for their inputs and outputs
- SystemVerilog supports variable bit widths through parameters (similar to templates in C++)
- This allows us to instantiate different versions of our modules and increases reuse!
- Example on the right: Parametrized adder with carry in and carry out

adder.sv

parameter on which the number of input bits (`i_a` and `i_b`) and output bits (`o_c`) depend

default value of the parameter

```
module adder #(parameter N=64) ( input  logic [N-1:0] i_a,
                                 input  logic [N-1:0] i_b,
                                 input  logic         i_carry_in,
                                 output logic [N-1:0] o_c,
                                 output logic         o_carry_out );
  assign { o_carry_out, o_c } = i_a + i_b + i_carry_in;
endmodule
```

adder_tb.sv

```
module adder_tb;
  logic [63:0] l_a, l_b, l_c;
  logic l_carry_in, l_carry_out;

  adder #(64) dut( l_a,
                   l_b,
                   l_carry_in,
                   l_c,
                   l_carry_out );

  initial begin
    l_carry_in = 1'b0;
    l_a = 64'h0000_0000_0000_0000;
    l_b = 64'h0000_0000_0000_0000;
    #10;
    assert( l_c === 64'h0000_0000_0000_0000 );
    assert( l_carry_out === 1'b0 );

    // TODO: add additional test cases
  end
endmodule
```

always_comb

- Complex combinational logic: `always_comb`
- Respective statements inside following block reevaluated every time a right-hand-side signal of an assignment changes
- Alias of `always @(*)`, similar syntax will become important when we want to change our signals based on the clock in the next weeks
- One has to remember to put a `begin` and `end` if more than one statement should be part of the block! Similar to curly brackets in if-else clauses when programming C

SystemVerilog Module

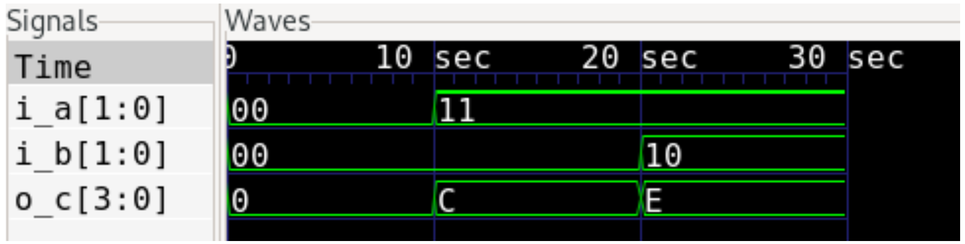
```
/**
 * An example module using always_comb.
 *
 * @param i_a four-bit input A.
 * @param i_b four-bit input B.
 * @param o_c four-bit output C.
 **/
module example_always_comb( input  logic [1:0] i_a,
                            input  logic [1:0] i_b,
                            output logic [3:0] o_c );

  always_comb
  begin
    o_c[1:0] = i_a[1:0] & i_b[1:0];
    o_c[3:2] = i_a[1:0] | i_b[1:0];
  end
endmodule
```

required since we'd like to have two statements in the block

blocking assignments; don't use `assign` syntax inside of `always_comb`

Simulation



case

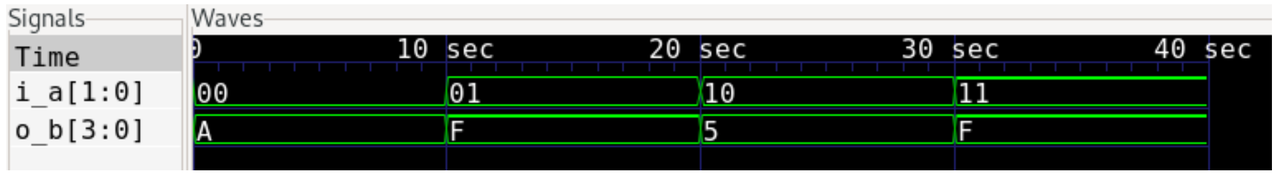
- SystemVerilog's `case` statement allows us to check if expression matches one in the given list and branch accordingly
- Only considered to be combinational logic if all possible cases are covered
- Recommendation: use the `default` statement!

SystemVerilog Module

```
/**
 * An example module using case.
 *
 * @param i_a two-bit input A.
 * @param o_b four-bit output B.
 **/
module example_case( input  logic [1:0] i_a,
                     output logic [3:0] o_b );
  always_comb
  begin
    case( i_a )
      2'b00: o_b[3:0] = 4'b1010;
      2'b10: o_b[3:0] = 4'b0101;
      default: o_b[3:0] = 4'b1111;
    endcase
  end
endmodule
```

begin/end is not required here

Simulation



always_ff

- Example on the right shows modulo-k counter which has the counter's value as output
- Uses always_ff block which implies flip-flops
- Here: Positive edge-triggered (i_clk or i_reset)

```
/**
 * Counter which has the number of elapsed clock cycles as output.
 *
 * Be aware that this counter is subject to output glitches
 * if more than one bit of o_count changes.
 *
 * @param i_clk input clock.
 * @param i_reset reset signal which resets the current count to zero.
 * @return o_count number of elapsed clock cycles.
 **/
module counter #(parameter N=8) ( input  logic         i_clk,
                                  input  logic         i_reset,
                                  output logic [N-1:0] o_count );
  // current and next value of the counter
  logic [N-1:0] m_count;

  // on positive edge of input clock or reset
  // either reset current counter or assign next value
  always_ff @(posedge i_clk, posedge i_reset)
    begin
      if( i_reset == 1'b1 )
        begin
          m_count <= 0;
        end
      else
        begin
          m_count <= m_count + 1;
        end
    end

  // current counted cycles as output
  assign o_count = m_count;
endmodule
```